

****Labsheet-6****

****Disha Samanta****

****CU24250227****

****Roll No- 20****

****Section- B****

Labsheet-6

Disha Samanta

CU24250227

Roll No- 20

Section- B

1. Select a suitable dataset such as the House Price Prediction Dataset, Advertising Sales Dataset, or California Housing Dataset.

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

housing = fetch_california_housing(as_frame=True)

df = housing.frame

print(df.head())

```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	\
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	

	Longitude	MedHouseVal
0	-122.23	4.526
1	-122.22	3.585
2	-122.24	3.521
3	-122.25	3.413
4	-122.25	3.422

2. Import the required libraries including Pandas, NumPy, Matplotlib, and Scikit-learn, and load the dataset.

```
print("Shape of dataset:", df.shape)

print("\nColumns:")
print(df.columns)

print("\nDataset Information:")
print(df.info())

print("\nMissing Values:")
print(df.isnull().sum())

print("\nStatistical Summary:")
print(df.describe())
```

3. Perform basic data preprocessing by handling missing values, encoding categorical variables (if any), and selecting relevant features.

```
print("Missing values before handling:")
print(df.isnull().sum())

df = df.fillna(df.median(numeric_only=True))

print("\nMissing values after handling:")
print(df.isnull().sum())
```

```
Missing values before handling:
MedInc      0
HouseAge    0
AveRooms    0
AveBedrms   0
Population  0
AveOccup    0
Latitude    0
Longitude   0
MedHouseVal 0
dtype: int64
```

```
Missing values after handling:
MedInc      0
HouseAge    0
AveRooms    0
AveBedrms   0
Population  0
AveOccup    0
Latitude    0
Longitude   0
MedHouseVal 0
dtype: int64
```

4. Split the dataset into training and testing sets using an appropriate train-test ratio (e.g., 80:20).

```
X = df.drop("MedHouseVal", axis=1)
```

```
y = df["MedHouseVal"]
```

```
print("Features:")
print(X.head())
```

```
print("\nTarget:")
print(y.head())
```

Features:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	\
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	

Longitude

0	-122.23
1	-122.22
2	-122.24
3	-122.25
4	-122.25

Target:

0	4.526
1	3.585
2	3.521
3	3.413
4	3.422

Name: MedHouseVal, dtype: float64

5. Train a Linear Regression model using the training dataset.

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
```

```
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

Training data: (16512, 8)

Testing data: (4128, 8)

6. Use the trained model to predict values for the testing dataset.

```
model = LinearRegression()

model.fit(X_train, y_train)

print("Model training completed.")
```

Model training completed.

7. Visualize the regression line and compare the actual and predicted values using appropriate plots

```
y_pred = model.predict(X_test)

print("Actual values:")
print(y_test.head())

print("\nPredicted values:")
print(y_pred[:5])
```

Actual values:

20046	0.47700
3024	0.45800
15663	5.00001
20484	2.18600
9814	2.78000

Name: MedHouseVal, dtype: float64

Predicted values:

[0.71912284 1.76401657 2.70965883 2.83892593 2.60465725]

Evaluate the model using performance metrics such as Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2 Score.

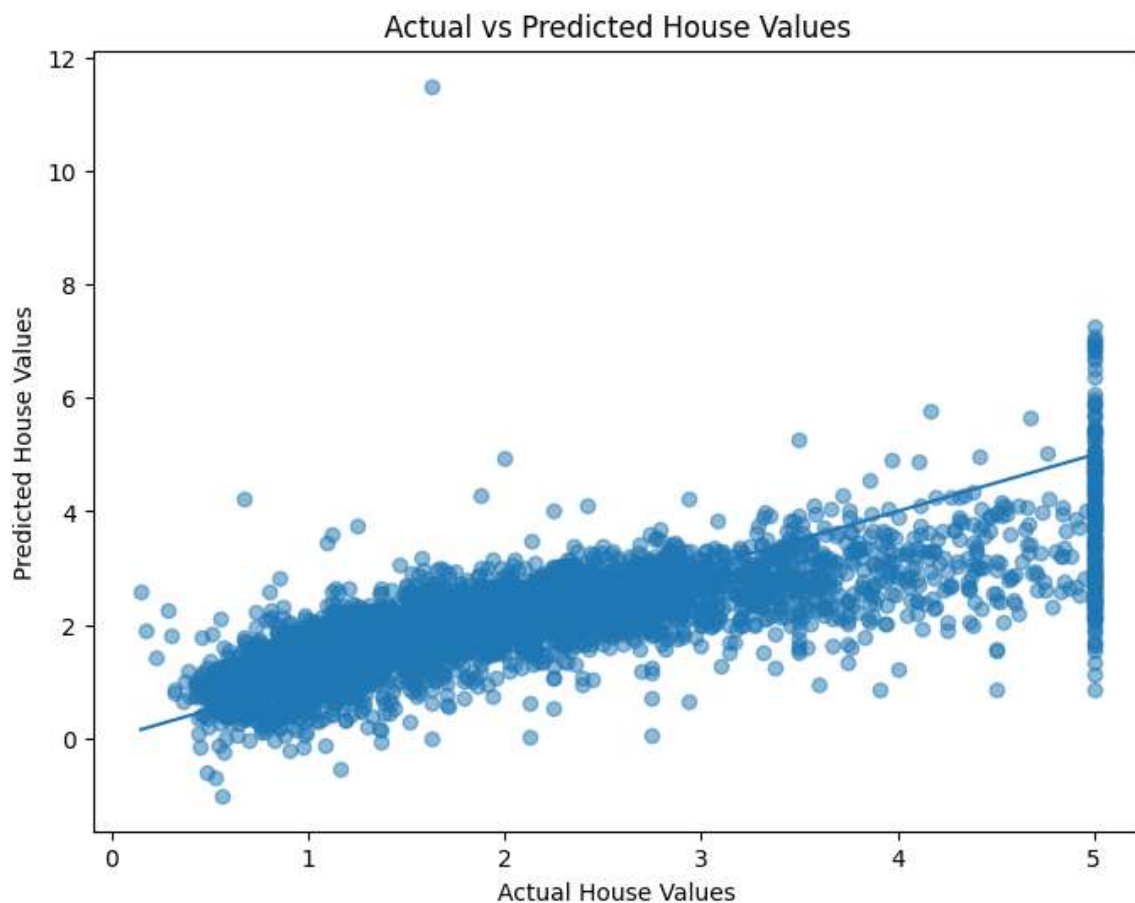
```
plt.figure(figsize=(8, 6))

plt.scatter(y_test, y_pred, alpha=0.5)

plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()]
)

plt.xlabel("Actual House Values")
plt.ylabel("Predicted House Values")
plt.title("Actual vs Predicted House Values")

plt.show()
```



9. Analyze the regression coefficients to understand the impact of each independent variable on the target variable.

```
mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, y_pred)

print("Mean Absolute Error (MAE):", mae)
print("Mean Squared Error (MSE):", mse)
print("Root Mean Squared Error (RMSE):", rmse)
print("R2 Score:", r2)
```

```
Mean Absolute Error (MAE): 0.5332001304956553
Mean Squared Error (MSE): 0.5558915986952444
Root Mean Squared Error (RMSE): 0.7455813830127764
R2 Score: 0.5757877060324508
```

10. Interpret the results and discuss the suitability of Linear Regression for the selected prediction problem.

```
coefficients = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_
})

coefficients["Absolute_Coefficient"] = coefficients["Coefficient"].abs()

coefficients = coefficients.sort_values(
    by="Absolute_Coefficient",
    ascending=False
)

print(coefficients)
```

	Feature	Coefficient	Absolute_Coefficient
3	AveBedrms	0.783145	0.783145
0	MedInc	0.448675	0.448675
7	Longitude	-0.433708	0.433708
6	Latitude	-0.419792	0.419792
2	AveRooms	-0.123323	0.123323
1	HouseAge	0.009724	0.009724
5	AveOccup	-0.003526	0.003526
4	Population	-0.000002	0.000002